

Course Code: CSE 4011	Course Name: Intelligent Developer Tools and AI DevOps Workflows	
Credits: 3 (2-0-2)	Contact Hours: 2-hour Theory & 2 hours lab per week	
Batch: 2023-27, 7 th Sem Academic Year: 2026-27	Semester Duration: 27 July 2026 to 24 Nov 2026	
Course Faculty: Dr. Kiran Khatter Email: kiran.khatter@bmu.edu.in Contact No: 9899960660 Office: 62, IV Floor, E2 Building Mr. Naveen Singh Email: ns340347@gmail.com Contact No: 9560818972 Office: 62, IV Floor, E2 Building	Course Coordinator: Dr. Kiran Khatter Email: kiran.khatter@bmu.edu.in Contact No: 9899960660 Office: 62, IV Floor, E2 Building	
Aim of the course: This course is intended to develop students' understanding of AI-powered DevOps tools and practices for modern software engineering. It focuses on the application of Large Language Models (LLMs), prompt engineering, intelligent code analysis, Retrieval-Augmented Generation (RAG), AI-assisted testing, and workflow automation throughout the software development lifecycle.		
Course Overview and Context: The integration of Artificial Intelligence with DevOps is transforming the way software is developed, tested, and delivered. This course introduces students to Large Language Models (LLMs), prompt engineering, semantic code understanding, Retrieval-Augmented Generation (RAG), AI-powered testing, and intelligent workflow automation. Students will acquire practical skills in AI-assisted code analysis, AI-enhanced CI/CD, and modern DevOps practices using state-of-the-art AI tools. By the end of the course, students will be able to design and implement AI-augmented software development pipelines.		

Course Philosophy

This course is delivered using a **Project-Based Learning (PjBL)** approach, where students progressively design, develop, and deploy an **AI-powered DevOps solution** throughout the semester. The course bridges modern software engineering, DevOps practices, and Generative AI by integrating theoretical concepts with hands-on implementation using state-of-the-art AI-assisted development tools. Students gain practical experience in leveraging Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), intelligent code assistants, and AI-driven automation to improve software development, testing, deployment, and maintenance workflows.

Throughout the course, students progressively build an end-to-end AI-enabled software engineering solution by integrating concepts of:

- AI-Assisted Software Development and Prompt Engineering
- Semantic Code Search and Repository Intelligence
- Large Language Models (LLMs) for Code Understanding and Generation
- Retrieval-Augmented Generation (RAG) for Intelligent Code Retrieval
- AI-Driven Testing, Debugging, and Code Quality Assessment
- AI-Enhanced CI/CD and DevOps Workflow Automation

- Developer Productivity Tools (Sourcegraph, Cursor, Gemini CLI, V0.dev, Sweep.dev, etc.)
- Responsible AI, Ethics, Explainability, and Limitations in Software Engineering

By the end of the course, students will have developed a production-oriented AI DevOps workflow that demonstrates the integration of modern AI technologies into the software development lifecycle, preparing them for next-generation software engineering and DevOps roles in industry.

Criterion Requirement Faculty Justification (Tick & Elaborate)

Application-Oriented Learning Students apply concepts to build a working

system rather than reproduce theory ✓ ☐ Yes ☐ No

Demonstrable Outcome Course leads to a tangible output such as a

working web application deployed or

demonstrated

✓ ☐ Yes ☐ No

Team Feasibility Project can be completed in teams of 3–5 with

clear role distribution ✓ ☐ Yes ☐ No

Assessability Learning can be evaluated through structured

reviews, demos, and viva ✓ ☐ Yes ☐ No

Course Outcomes (CO):

At the end of the course, the students should be able to do the following:

	Course Outcome Statement	Mapped Component	Associated Activities / Assessment
CO1	Understand foundational concepts and tools that enable code intelligence using AI	Employability	→ Intensive theory sessions on AI DevOps foundations, trends in AI-assisted software engineering, and developer productivity tools. → In-class demonstrations of tools like Code Llama, StarCoder2, DeepSeek Coder, and access via Hugging Face/Ollama. → Case discussions on the evolution of AI in software development.
CO2	Apply tools for intelligent code search, navigation, testing, and automation	Skill Development	→ Hands-on lab sessions with tools such as Sourcegraph OSS, CodiumAI, Codeium, GitHub Actions, and Sweep.dev. → Practical exercises on semantic search, cross repository navigation, test generation, bug detection, and CI/CD pipeline automation. → Implementation of AI-assisted workflows using VS Code extensions and GitHub integrations.

CO3	Create AI-augmented workflows for software engineering	Entrepreneurship, Skill Development	→ Projects involving the use of RAG (Retrieval Augmented Generation) pipelines using LangChain, ChromaDB, and FAISS. → Design of intelligent bots for code Q&A, project indexing, and documentation-based test generation. → Final course project integrating tools across the AI DevOps pipeline to solve real-world development problems.
-----	--	-------------------------------------	---

Prerequisite: if any

Basic knowledge of programming, familiarity with Git and version control, and understanding of software development workflows.

Topics of the course:

Topics	No. of Sessions	CO
Understand the scope and trends of AI DevOps	4 (3 Th + 1 Lab)	CO1

and its relevance to modern software engineering		
Apply prompt engineering techniques to AI code models for completion and refactoring	4 (3 Th + 1 Lab)	CO1
Evaluate the capabilities of different code models (e.g., Code Llama, StarCoder2)	2 (1 Th + 1 Lab)	CO1
Set up Sourcegraph for semantic code navigation and query formulation	5 (4 Th + 1 Lab)	CO2
Use LLM-powered tools for code understanding and explanation across repositories	3 (2 Th + 1 Lab)	CO2
Analyze RAG-based architectures and their accuracy implications in code-centric retrieval	4 (2 Th + 2 Lab)	CO2
Build and refine a code Q&A bot using LangChain or LlamaIndex with real project repositories	4 (3 Th + 1 Lab)	CO2
Evaluate AI-based testing tools (e.g., CodiumAI, Codeium) for coverage and usefulness	3 (2 Th + 1 Lab)	CO3
Apply GitHub Actions for workflow automation with AI-enhanced CI/CD steps	4 (3 Th + 1 Lab)	CO3
Design end-to-end DevOps workflows using Sweep.dev and other AI-based automation tools	3 (2 Th + 1 Lab)	CO3
Integrate RAG pipelines into DevOps toolchains for intelligent code retrieval and resolution	4 (2 Th + 2 Lab)	CO3
Reflect on AI ethics, limitations, and interpretability in developer tools	3 (2 Th + 1 Lab)	CO1
Applications of Gemini CLI, V0.dev, Cursor, and Sourcegraph	5 (3 Th + 2 Lab)	CO3
32 Theory sessions + 16 Lab sessions = 64 total contact hours		

CO/PO Mapping: 3 (Strongest Mapping), 1 (Weak Mapping)

[illegible]

CO1	3	2	1	1	2			2				3	1	2		
CO2	2	3	3	3	3				2	1	2	2	3	2		
CO3	2	3	3	2	3	1		2	3	2	3	2	2	3		

Learning Resources:

Textbooks / Key References:

- Flanagan, D. (2020). JavaScript: The Definitive Guide. O'Reilly Media.
- Langchain Team. (2024). LangChain Documentation. <https://docs.langchain.com>
- LlamaIndex Contributors. (2024). LlamaIndex Documentation. <https://docs.llamaindex.ai>
- Sourcegraph Documentation. (2024). <https://docs.sourcegraph.com>
- CodiumAI Documentation. (2024). <https://www.codium.ai/learn>
- Codeium Documentation. (2024). <https://docs.codeium.com>
- Sweep.dev Documentation. (2024). <https://docs.sweep.dev>

Reference Links:

Due to the fast-evolving nature of AI-based development tools, the course will rely on a range of current and openly accessible materials including software documentation, GitHub repositories, blog posts, and tutorials. Links to relevant resources will be provided weekly via LMS.

Suggested Online Course:

<https://www.coursera.org/learn/devops-dataops-mlops-duke>

<https://www.coursera.org/learn/genai-for-devops-practitioners>

Experiential Learning Component (45 %):

The Experiential Learning Component of the Intelligent Developer Tools and AI DevOps (CSE 4011) Workflows course is designed to provide students with hands-on experiences that reinforce theoretical knowledge through practical application. Throughout the course, students will engage in project-based learning where they will be assigned practical projects that evolve from foundational code intelligence tools to advanced AI-augmented DevOps workflows. These projects, which span the entirety of the course, are structured into phase-wise evaluations to ensure consistent application and progression of skills.

The first project phase evaluation, scheduled for the week starting 21st September with a weightage of 30%, focuses on the fundamentals of AI-augmented code understanding and navigation. This includes setup and usage of tools such as Sourcegraph OSS and VS Code AI extensions (e.g., TabNine), prompt engineering for intelligent code completion and refactoring, and hands-on interaction with open-source AI code models. Students will be assessed on their ability to configure tools, navigate codebases, engineer prompts, and document their implementation.

The second project phase, starting the week of 5th October and also weighted at 30%, emphasizes the integration of multiple tools and concepts including LangChain, LlamaIndex, and GitHub Actions. Students will work on building a RAG-based code Q&A bot, automate DevOps workflows using GitHub Actions and Sweep.dev, and integrate AI tools like CodiumAI and Codeium for intelligent test generation and bug detection. Evaluation will focus on effective integration of AI tools, workflow coherence, automation accuracy, and overall quality of code and documentation.

Assessment Pattern: The final grade will be determined by the marks or grades earned during the project's phase-wise evaluations and the end-term assessment. Grading will be conducted using the relative grading method outlined in the university's academic regulations. To be eligible for grading, students must achieve a minimum of 40% of the total marks upon completing all assessments listed in the table below:

Evaluation Component	Weightage (%)	Evaluation Schedule	Rubrics
-----------------------------	----------------------	----------------------------	----------------

A1 – Project Phase Evaluation 1	30%	week starting 21st September	Focus: Knowledge and application of AI-augmented code understanding and navigation. Assessment (as per rubrics): – Synopsis Finalization / Problem Definition & Experience Design (5%) [will be evaluated in the week starting from 24th August] - Prompt engineering for code completion and refactoring (5%) - Source graph setup and semantic code navigation (10%) - Tool configuration and code exploration (5%) - Documentation and accessibility of implementation (5%) Criteria: - C1: Problem Definition - C2: Objectives and Outcomes - C3: Methodology - C4: Feasibility and Resource Planning - C5: Individual Understanding of Project Student Submissions: ● One-page Project Charter / Synopsis ● Project Timeline ● Team Responsibility Matrix
--	-----	------------------------------	--

A2 – Project Phase Evaluation 2	30%	Week scheduled for Mid-Term Examination (5 Oct-9 Oct)	Focus: Integration of AI tools, code retrieval, and automation workflows. Assessment (as per rubrics): - RAG-based Q&A bot and use of Lang Chain or Llama Index (10%) - GitHub Actions CI/CD with Sweep.dev for automation(10%) - Test generation using Codium AI/Codeium (5%) - Integration quality and documentation (5%) Criteria: - C6: Progress vs Plan - C7: Technical Understanding - C8: Problem Solving - C9: Documentation (Working) - C10: Individual Contribution Student Submissions: ● Updated Project Charter / Synopsis (if revised) ● Revised Project Timeline (progress vs plan) ● Working Prototype / Code Demonstration ● Brief Progress Report (1–2 pages or
--	-----	---	--

			demo-based evidence)
A3 – End Term Evaluation (Final Project)	40%	As per Academic Calendar / Datesheet	Focus: Comprehensive application of AI-powered DevOps tools and intelligent developer workflows. Assessment (as per rubrics): - Integration of retrieval and automation pipelines (15%) - Use of multiple tools with coordinated flow (10%) - Custom prompt design, code search, and evaluation (5%) - Functional completeness and innovation (5%) - Presentation, team contribution, and usability (5%) Criteria: C11: Final Product / Solution C12: Technical Depth C13: Innovation / Application C14: Presentation & Communication C15: Individual Contribution & Learning Student Submissions: ● Final Functional Application / Product ● Source Code Repository (GitHub or equivalent) ● Final Project Report (concise, structured) ● Final Presentation PPT & Demonstration ● Final Group/Self Contribution Matrix

Student Responsibilities:

- Attend lectures and do the work Lab Assignments as per instructions.
- Participate in the discussions/assignments held during classes.
- Check announcements on the LMS and emails regularly.
- Submit the assigned task on time.
- Regularly check marks on the LMS to ensure they are up to date.
- Participate in class and take necessary actions to grasp the material. Asking questions is encouraged.
- Communicate any concerns by speaking directly with the instructor.

Attendance Policy: Students are expected to attend classes regularly. Failure to follow the classes regularly and adhere to the expected attendance percentage will result in losing quiz/lab marks and a reduction of the grade as per the University's grading policy.

Recourse Examination Policy: In case a student fails the course, a one-time recourse is permitted as per the academic regulations of the University. Recourse is allowed only for the End Semester examination.

Make-up policy: No make-up exam will be conducted for unexcused absences. The faculty needs to be informed in advance in case the student is not going to appear for any evaluation component, and it is at the discretion of the faculty to sanction makeup for an evaluation component.

Behavior Expectations: No mobile phones and other distractive gadgets are permitted in the class.

Academic Dishonesty/Cheating/Plagiarism: Plagiarism and dishonesty in any form in any evaluation component will lead to appropriate disciplinary action.